SOLID Programming principles

Introduction to SOLID principles of object-oriented programming.

Introduction to SOLID Programming

SOLID is an acronym that stands for five key principles of object-oriented programming and design: Single responsibility, Open-closed, Liskov substitution, Interface segregation and Dependency inversion. Following these principles helps developers write more maintainable, flexible and reusable code.

SOLID Acronym

- **S: Single Responsibility Principle**

  A class should have only one reason to change - it should only have one responsibility.

- **O: Open Closed Principle**

  Software entities should be open for extension but closed for modification.

- **L: Liskov Substitution Principle**

  Subtypes must be substitutable for their base types.

- **I: Interface Segregation Principle**

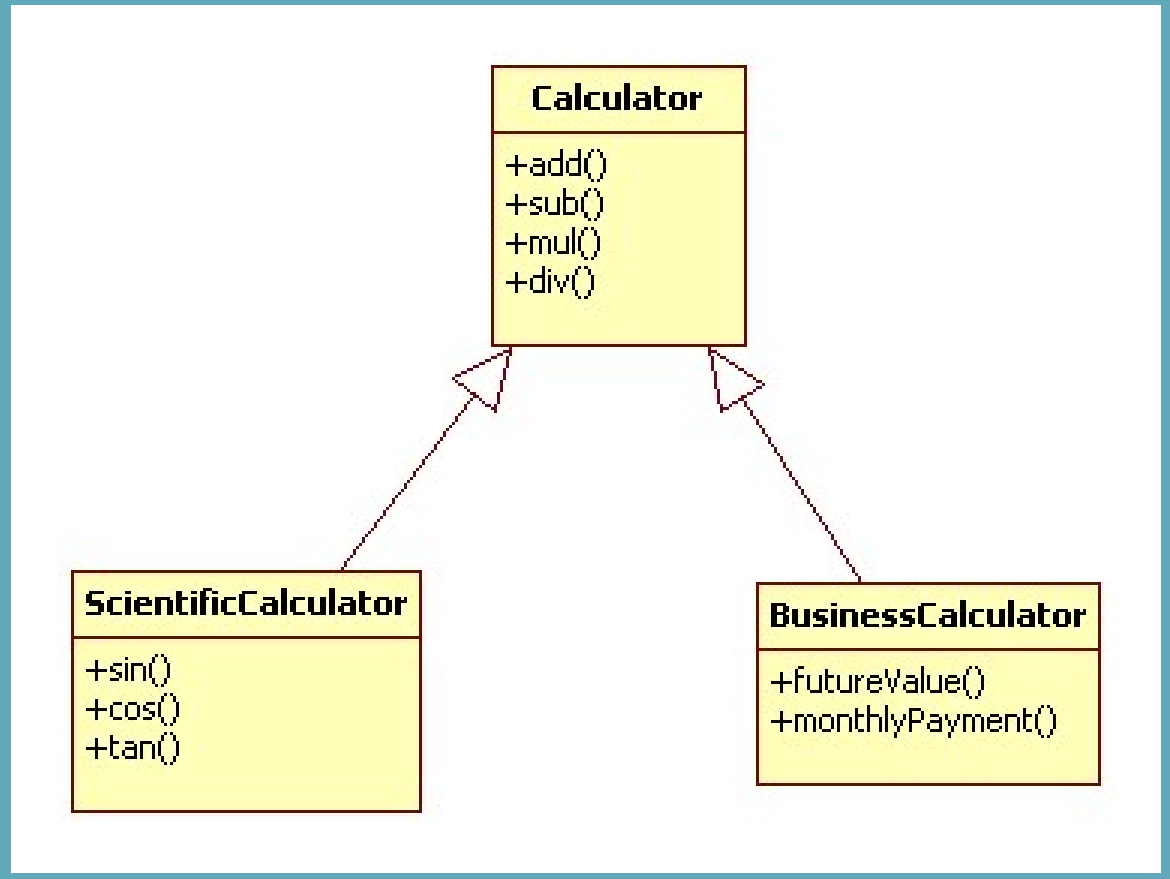
  Clients should not be forced to depend on interfaces they do not use.

- **D: Dependency Inversion Principle**

  Depend on abstractions, not on concretions.

Single Responsibility Principle

The single responsibility principle is an important concept in object-oriented programming. It states that a class should have only one reason to change. This helps to keep classes focused and reduces complexity in code. The single responsibility principle is one of the five SOLID principles of object-oriented design.

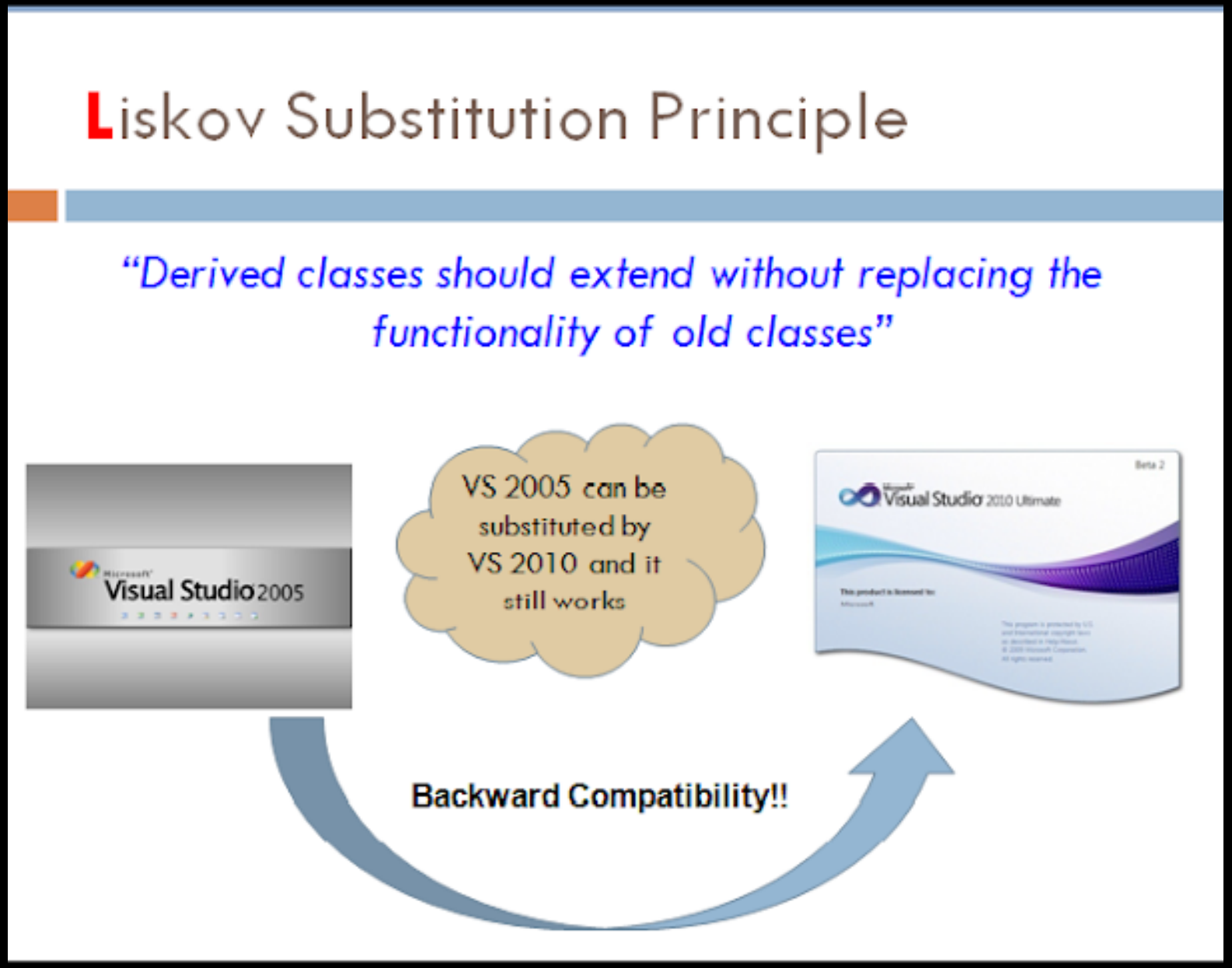


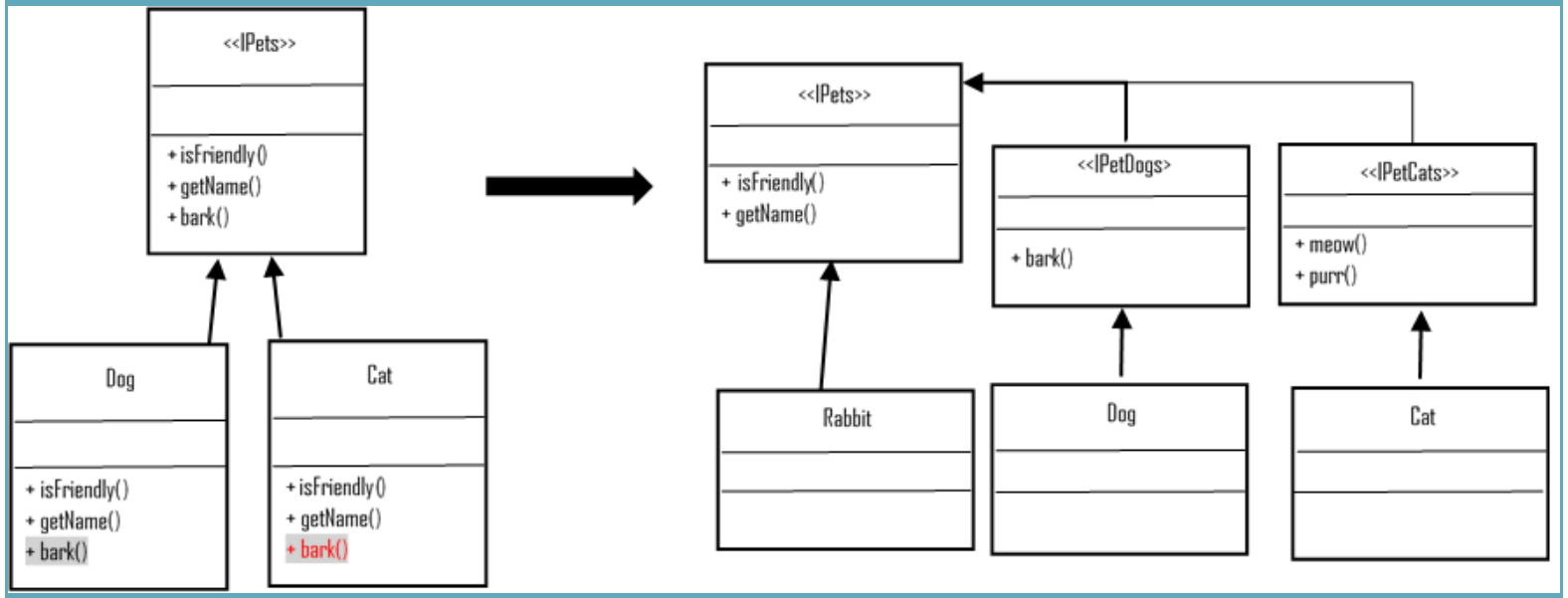
Open Closed Principle

The open-closed principle states that classes should be open for extension but closed for modification. This allows new features to be added without modifying existing code. By designing classes with interfaces and using abstraction, new subclasses can extend functionality while the base class remains unchanged.

Liskov Substitution Principle

The Liskov Substitution Principle is an important concept in object-oriented programming. It states that derived classes should be substitutable for their base classes without changing the correctness of the program. This allows a programmer to use objects of derived classes in place of objects of base classes and ensures that a program continues to function properly.





Interface Segregation Principle

The Interface Segregation Principle states that clients should not be forced to depend on interfaces they don't use. For example, if a class only needs one method from an interface, it's better to split that interface into multiple smaller interfaces so the class only depends on what it needs.

Dependency Inversion Principle

The Dependency Inversion principle is an important concept in object-oriented design that states that high-level modules should not depend on low-level modules. Instead, both should depend on abstractions. This decouples the modules and makes the high-level modules more reusable and resistant to changes in the low-level modules.

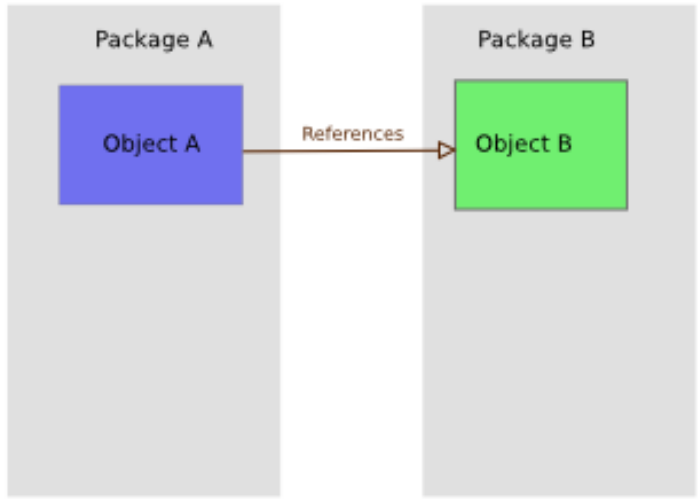


Figure 1

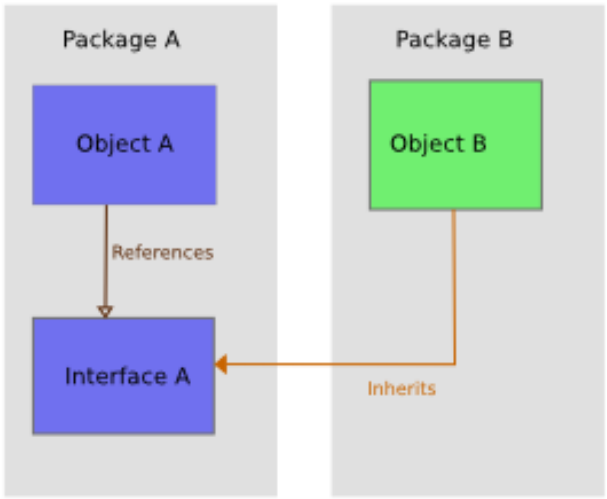


Figure 2

Benefits of SOLID Programming

- ## Increased code maintainability and readability

  Following SOLID principles like Single Responsibility and Open/Closed principle lead to more modular and readable code.

- ## Loose coupling between components

  Principles like Dependency Inversion promote loose coupling by depending on abstractions rather than concrete implementations.

- ## Higher code quality and less bugs

  Well-structured and modular code reduces complexity and helps prevent bugs and unintended side effects.

- ## Easier to extend and add new features

  Open/Closed principle allows extending behavior without modifying existing code, making new features easy to add.

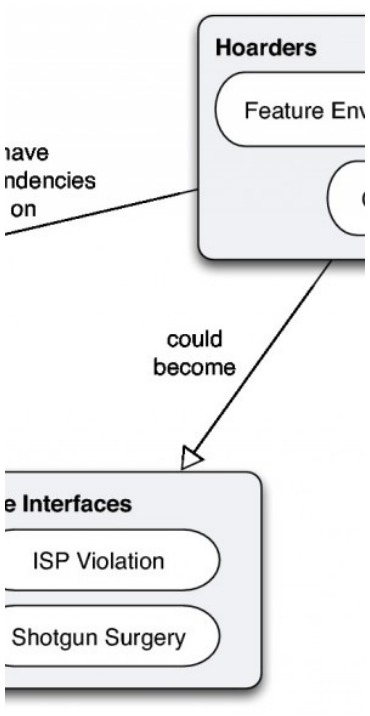
- ## Code more testable

  Following SOLID facilitates unit testing by promoting loose coupling and separation of concerns.

Common SOLID Violations

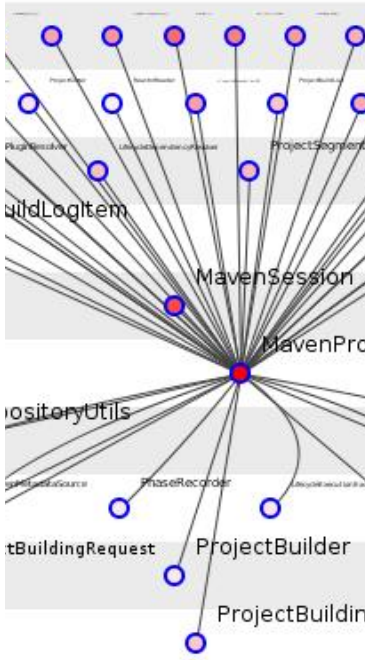
Tight Coupling

Image of two classes that depend too much on each other's implementation details.



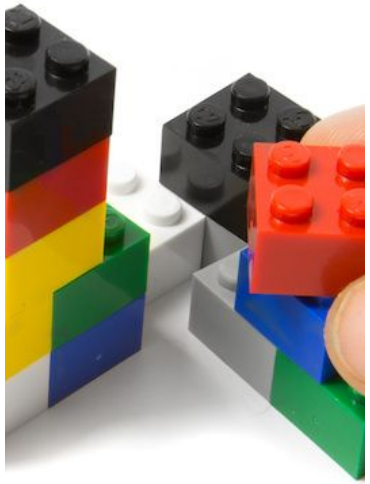
Rigidity

Image of inflexible class hierarchy that is hard to change.



Fragility

Image of class doing too many things, hard to change without breaking other things.

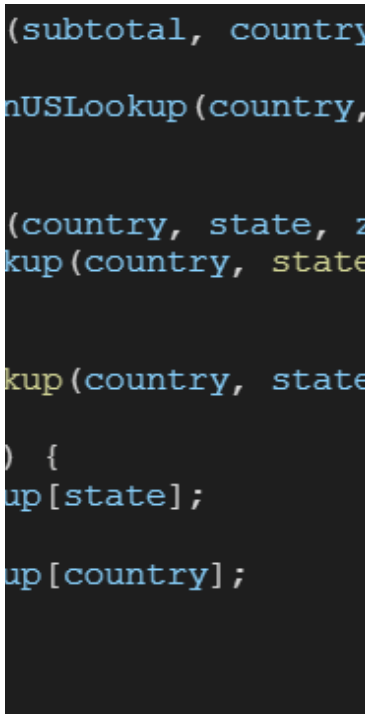


Immobility

Image of reusable component tied to a specific application making it hard to reuse.

Opacity

Image of complex class doing too many things making it hard to understand.



Needless Repetition

Image of duplicated code across classes instead of shared parent class.

Adopting SOLID Principles

- **Start with Single Responsibility Principle**

  Ensure each class and method has a single clear purpose

- **Apply Open/Closed Principle**

  Design modules to be open for extension but closed for modification

- **Use Liskov Substitution Principle**

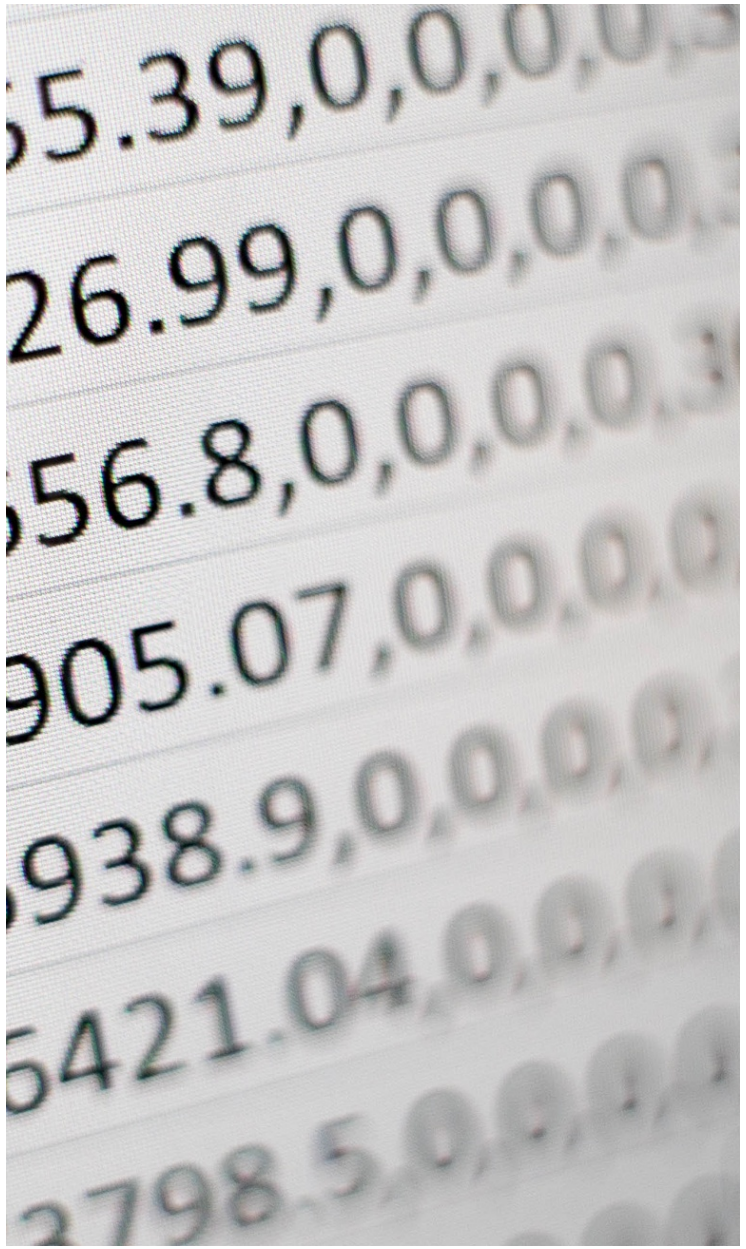
  Subtypes should be substitutable for their base types

- **Prefer Interface Segregation Principle**

  Split interfaces into smaller ones with specific client needs

- **Depend on Abstractions Principle**

  High-level modules shouldn't depend on low-level implementation details



Conclusion

The SOLID principles provide guidelines for writing object-oriented code that is flexible, maintainable, and extendable. Following these principles allows developers to build software that can easily evolve over time. Some key benefits of using SOLID principles are increased code readability and reduced coupling.